STATISTICS & MACHINE LEARNING · LESSON 28 OF 28

ML Capstone: End-to-End Churn Prediction and Retention Strategy

Put the whole ML track together. Frame the churn problem, build leak-free features, compare baselines and models with cross-validation, tune the winner, choose a profit-maximising threshold under a budget, explain predictions, design segment-level retention actions with an A/B test, and write the model card, deployment plan and executive summary for your portfolio.

ML capstone · ✦ scikit-learn 1.7+ · LightGBM 4 · SHAP · Advanced · ⏱ 180 min

By the end of this lesson you can

- Frame a churn problem with a decision, metric, baseline and constraints
- Build a reproducible pipeline with engineered features and compare models with cross-validation
- Tune and evaluate the final model on an untouched test set
- Choose the operating threshold from campaign economics and capacity
- Explain, document and plan deployment and measurement of the model

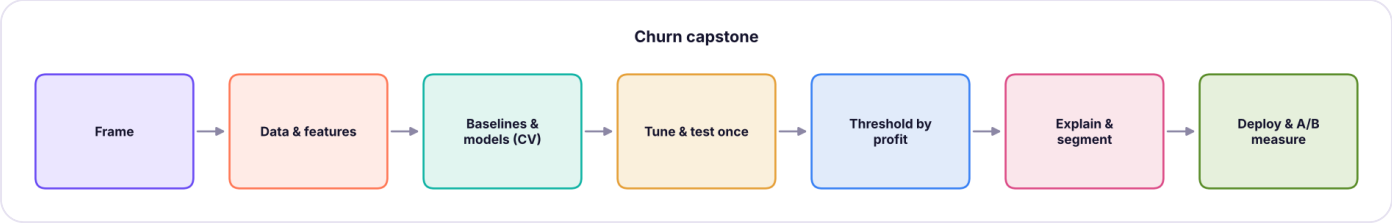
Datasets used

`telecom_churn.csv`

1. Frame the problem

Brief. A telecom operator loses about a quarter of its customers every year. The retention team has a monthly budget to contact **500 customers** with an offer worth ₹300 (cost ₹300 per contacted customer, accepted or not). A saved customer is worth about **₹4,000** of future margin. Historically, **35%** of churners who receive the offer stay. **Which customers should get the offer?**

Item	Decision
Decision	Rank customers each month; contact the top N within budget
Target	`churn` = Yes (left within the observation window)
Unit	Customer
Metric	Expected campaign profit; also ROC-AUC and recall in the top N
Baseline	Current rule: month-to-month customers with 3+ support calls
Constraints	≤ 500 contacts; explainable reasons for agents; no use of sensitive attributes for targeting



Capstone workflow.

2. Data and features

Python

```
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 160); pd.set_option("display.max_columns", 20)
tc = pd.read_csv("telecom_churn.csv")
y = (tc.churn == "Yes").astype(int)
print(tc.shape, f"churn rate {y.mean():.3f}")

def add_features(df):
    out = df.copy()
    out["is_new"] = (out.tenure_months <= 6).astype(int)
    out["charge_per_tenure_month"] = out.total_charges / (out.tenure_months + 1)
    out["price_change_proxy"] = out.monthly_charges - out.charge_per_tenure_month     #
charges now vs lifetime average
    out["high_support"] = (out.support_calls >= 3).astype(int)
    out["m2m_and_new"] = ((out.contract == "Month-to-month") & (out.tenure_months <=
12)).astype(int)
    return out.drop(columns=["customer_id", "churn"])
X = add_features(tc)
print(X.columns.tolist())
```

OUTPUT

```
(3000, 10) churn rate 0.257
['senior_citizen', 'tenure_months', 'contract', 'internet_service', 'payment_method',
```

```
'monthly_charges', 'total_charges', 'support_calls', 'is_new',
'charge_per_tenure_month', 'price_change_proxy', 'high_support', 'm2m_and_new']
```

All features are known **before** the churn event (tenure, plan, charges, recent support calls), so there is no leakage. `senior_citizen` is kept for fairness checks but excluded from targeting features (a business and ethics choice).

Python

```python
from sklearn.model_selection import train_test_split
X_model = X.drop(columns=["senior_citizen"])
Xtr, Xte, ytr, yte, idx_tr, idx_te = train_test_split(X_model, y, tc.index, test_size=0.25,
stratify=y, random_state=2026)
print(f"train {len(Xtr)} | test {len(Xte)} (locked away until the end)")
```

OUTPUT

```
train 2250 | test 750 (locked away until the end)
```

3. Baselines and candidate models

Python

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
import lightgbm as lgb
num = [c for c in X_model.columns if X_model[c].dtype != object and str(X_model[c].dtype)
!= "str"]
cat = [c for c in X_model.columns if c not in num]
pre = ColumnTransformer([("num", StandardScaler(), num), ("cat",
OneHotEncoder(handle_unknown="ignore"), cat)])
cv = StratifiedKFold(5, shuffle=True, random_state=0)
candidates = {
    "dummy (base rate)": DummyClassifier(strategy="prior"),
    "logistic regression": LogisticRegression(max_iter=3000, C=0.5),
    "random forest": RandomForestClassifier(500, min_samples_leaf=20, max_features=0.5,
random_state=0, n_jobs=1),
    "HistGradientBoosting": HistGradientBoostingClassifier(learning_rate=0.05,
max_iter=200, max_leaf_nodes=15, min_samples_leaf=40, random_state=0),
    "LightGBM": lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03, num_leaves=15,
min_child_samples=40, subsample=0.8, subsample_freq=1, colsample_bytree=0.8, verbose=-1,
random_state=0, n_jobs=1),
}
rows = []
for name, est in candidates.items():
    r = cross_validate(Pipeline([("pre", pre), ("model", est)]), Xtr, ytr, cv=cv, scoring=
["roc_auc", "average_precision", "neg_log_loss"])
    rows.append({"model": name, "ROC-AUC": r["test_roc_auc"].mean(), "± sd":
r["test_roc_auc"].std(), "PR-AUC": r["test_average_precision"].mean(), "log loss": -
r["test_neg_log_loss"].mean()})
board = pd.DataFrame(rows).round(3).sort_values("ROC-AUC", ascending=False);
print(board.to_string(index=False))
rule = ((Xtr.contract == "Month-to-month") & (Xtr.support_calls ⩾ 3)).astype(int)
from sklearn.metrics import recall_score, precision_score
print(f"business rule on train: flags {rule.mean():.1%} | recall {recall_score(ytr,
rule):.3f} | precision {precision_score(ytr, rule):.3f}")
```

OUTPUT

model	ROC-AUC	± sd	PR-AUC	log loss
logistic regression	0.815	0.036	0.624	0.440
random forest	0.803	0.033	0.590	0.456
LightGBM	0.796	0.031	0.576	0.469
HistGradientBoosting	0.793	0.031	0.571	0.476

```
   dummy (base rate)    0.500 0.000   0.257     0.570
business rule on train: flags 8.5% | recall 0.171 | precision 0.516
```

4. Tune the leading models

Python

```python
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import loguniform, randint, uniform
lr_search = GridSearchCV(Pipeline([("pre", pre), ("model",
LogisticRegression(max_iter=3000))]), {"model__C": np.logspace(-2, 1, 8)}, cv=cv,
scoring="roc_auc").fit(Xtr, ytr)
gb_search = RandomizedSearchCV(Pipeline([("pre", pre), ("model",
lgb.LGBMClassifier(verbose=-1, random_state=0, subsample_freq=1, n_jobs=1))]),
    {"model__n_estimators": randint(100, 600), "model__learning_rate": loguniform(0.01,
0.1), "model__num_leaves": randint(4, 32), "model__min_child_samples": randint(20, 120),
     "model__subsample": uniform(0.6, 0.4), "model__colsample_bytree": uniform(0.5, 0.5),
"model__reg_lambda": loguniform(0.01, 10)},
    n_iter=20, cv=cv, scoring="roc_auc", random_state=0, n_jobs=1).fit(Xtr, ytr)
print(f"logistic best C {lr_search.best_params_['model__C']:.3g} | CV AUC
{lr_search.best_score_:.3f}")
print(f"LightGBM best CV AUC {gb_search.best_score_:.3f} | params { {k.split('__')[1]:
(round(float(v), 3) if isinstance(v, float) else v) for k, v in
gb_search.best_params_.items()} }")
```

OUTPUT

```
logistic best C 0.193 | CV AUC 0.816
LightGBM best CV AUC 0.812 | params {'colsample_bytree': 0.659, 'learning_rate':
0.046, 'min_child_samples': 71, 'n_estimators': 180, 'num_leaves': 4, 'reg_lambda':
3.343, 'subsample': 0.706}
```

Both are close. Prefer the **simpler, more explainable** model unless the complex one is clearly better: we pick the tuned logistic regression if its CV AUC is within 0.005 of LightGBM, otherwise LightGBM.

Python

```python
from sklearn.metrics import roc_auc_score, average_precision_score
final = lr_search.best_estimator_ if lr_search.best_score_ ⩾ gb_search.best_score_ - 0.005
else gb_search.best_estimator_
final_name = "logistic regression" if final is lr_search.best_estimator_ else "LightGBM"
p_test = final.predict_proba(Xte)[:, 1]
print(f"chosen: {final_name} | TEST ROC-AUC {roc_auc_score(yte, p_test):.3f} | PR-AUC
{average_precision_score(yte, p_test):.3f} | base rate {yte.mean():.3f}")
```

OUTPUT

```
chosen: logistic regression | TEST ROC-AUC 0.793 | PR-AUC 0.579 | base rate 0.257
```

The test set was used **once**, after all choices were made, so this is an honest estimate.

5. Choose how many customers to contact

Expected profit for contacting customer *i* with churn probability p_i:

$$E[\text{profit}_i] = p_i \times 0.35 \times ₹4{,}000 - ₹300$$

So contacting pays off only when $p_i > 300 / (0.35 \times 4{,}000) \approx 0.21$, provided the probabilities are well calibrated.

Python

```python
from sklearn.calibration import calibration_curve
fr, mp = calibration_curve(yte, p_test, n_bins=8, strategy="quantile")
print("calibration (predicted → observed):", [f"{a:.2f}→{b:.2f}" for a, b in zip(mp, fr)])
value, accept, cost = 4000, 0.35, 300
order = np.argsort(-p_test); ys = yte.to_numpy()[order]; ps = p_test[order]
scale = 500 / 750                                                    # the test set is
750 of 3,000 customers: scale the 500 budget
res = []
for n in [25, 50, 75, 100, 125, 150, 200, 250, 300]:
    tp = ys[:n].sum(); profit = tp * accept * value - n * cost
    res.append({"contacts (test)": n, "≈ contacts (all)": int(n / 0.25), "churners
reached": int(tp), "precision": round(tp / n, 3), "recall": round(tp / ys.sum(), 3),
"profit ₹ (test)": int(profit)})
rt = pd.DataFrame(res); print(rt.to_string(index=False))
best = rt.loc[rt["profit ₹ (test)"].idxmax()]
print(f"most profitable: contact the top {int(best['contacts (test)'])} of 750 test
customers (≈ {int(best['≈ contacts (all)'])} of 3,000), cut-off probability ≈
{ps[int(best['contacts (test)']) - 1]:.2f}")
rand_tp = yte.mean() * 125; print(f"random 125 contacts would reach ≈ {rand_tp:.0f}
churners; profit ≈ ₹{int(rand_tp * accept * value - 125 * cost):,}")
```

OUTPUT

```
calibration (predicted → observed): ['0.02→0.03', '0.06→0.06', '0.10→0.11',
'0.16→0.20', '0.23→0.19', '0.32→0.34', '0.45→0.47', '0.71→0.65']
 contacts (test)  ≈ contacts (all)  churners reached  precision  recall  profit ₹
(test)
              25               100                18      0.720   0.093
17700
```

```
              50                200                36      0.720    0.187
35400
              75                300                51      0.680    0.264
48899
             100                400                63      0.630    0.326
58199
             125                500                75      0.600    0.389
67500
             150                600                88      0.587    0.456
78199
             200                800               110      0.550    0.570
94000
             250               1000               128      0.512    0.663
104200
             300               1200               141      0.470    0.731
107399
most profitable: contact the top 300 of 750 test customers (≈ 1200 of 3,000), cut-off
probability ≈ 0.25
random 125 contacts would reach ≈ 32 churners; profit ≈ ₹7,533
```

With the budget of 500 contacts per 3,000 customers (≈ 125 in the test set), the model reaches 75 churners against about 32 for random targeting (₹67,500 expected profit against about ₹7,500). Profit keeps rising up to about 300 test contacts (≈ 1,200 customers, cut-off probability ≈ 0.25, close to the theoretical 0.21), so the team has a case for a larger budget.

6. Explain and segment the targets

Python

```python
pre_f, mdl = final.named_steps["pre"], final.named_steps["model"]
names = pre_f.get_feature_names_out()
if hasattr(mdl, "coef_"):
    coefs = pd.Series(mdl.coef_[0], index=names).sort_values()
    print("largest effects (standardised log-odds):", pd.concat([coefs.head(4),
coefs.tail(4)]).round(3).to_dict())
targets = tc.loc[Xte.index[order[:125]]].assign(p_churn=ps[:125].round(2))
def action(r):
    if r.support_calls >= 3: return "Service recovery call + offer"
    if r.contract == "Month-to-month" and r.tenure_months <= 12: return "Annual-plan
upgrade discount"
    if r.internet_service == "Fiber": return "Fibre quality check + loyalty data pack"
    return "General loyalty offer"
targets["action"] = targets.apply(action, axis=1)
print(targets.action.value_counts().to_dict())
print(targets[["customer_id", "tenure_months", "contract", "support_calls",
"monthly_charges", "p_churn", "action"]].head(6).to_string(index=False))
```

OUTPUT

```
largest effects (standardised log-odds): {'cat__contract_Two year': -0.994,
'num__tenure_months': -0.877, 'cat__contract_One year': -0.174,
'cat__payment_method_Auto-debit': -0.17, 'num__monthly_charges': 0.168,
'cat__internet_service_Fiber': 0.222, 'num__support_calls': 0.406,
'cat__contract_Month-to-month': 1.165}
{'Annual-plan upgrade discount': 55, 'Service recovery call + offer': 34, 'Fibre
quality check + loyalty data pack': 27, 'General loyalty offer': 9}
customer_id  tenure_months       contract  support_calls  monthly_charges  p_churn
action
     T02973              2 Month-to-month              4            98.55     0.94
Service recovery call + offer
     T00025              2 Month-to-month              4            79.55     0.90
Service recovery call + offer
     T01423              3 Month-to-month              3            99.41     0.89
Service recovery call + offer
     T00671              4 Month-to-month              3            84.15     0.89
Service recovery call + offer
     T01945              2 Month-to-month              2            96.98     0.87
Annual-plan upgrade discount
     T00583              9 Month-to-month              3            82.31     0.87
Service recovery call + offer
```

Each contacted customer gets a **reason** (top contributions) and an **action** matched to that reason, which raises acceptance more than one generic offer.

Fairness check

Python

```
sen = tc.loc[Xte.index, "senior_citizen"].to_numpy(); flag = np.zeros(len(Xte), bool);
flag[order[:125]] = True
print(pd.DataFrame({"group": ["non-senior", "senior"], "customers": [int((sen == 0).sum()),
int((sen == 1).sum())],
                    "churn rate": [round(yte[sen == 0].mean(), 3), round(yte[sen ==
1].mean(), 3)],
                    "contacted": [round(flag[sen == 0].mean(), 3), round(flag[sen ==
1].mean(), 3)],
                    "recall": [round(recall_score(yte[sen == 0], flag[sen == 0]), 3),
round(recall_score(yte[sen == 1], flag[sen == 1]), 3)]}).to_string(index=False))
```

OUTPUT

```
      group  customers  churn rate  contacted  recall
non-senior        631       0.250      0.158   0.348
    senior        119       0.294      0.210   0.571
```

Contact rates and recall should be broadly in line with each group's churn rate. Here senior customers are contacted more often (21% vs 16%), which their higher churn rate partly explains, but their recall is also much higher (0.57 vs 0.35). The gap should be reviewed with the business before launch, for example by checking whether non-senior churners are being missed.

7. Measure with an experiment

The model predicts who will churn, not who can be **saved**. Prove the value with a randomised test:

1. Score all customers; take the top 600.
2. Randomly assign 500 to **treatment** (offer) and 100 to **holdout** (no offer).
3. After 60 days compare churn: treatment vs holdout, using a two-proportion test (ML lesson 6).
4. Incremental saves × ₹4,000 − cost = true campaign ROI. Next step: **uplift modelling**, which predicts who responds to the offer.

8. Model card and deployment plan

Python

```python
import json, sklearn
card = {"name": "Telecom churn propensity", "version": "1.0", "model": final_name,
"sklearn": sklearn.__version__,
        "features": list(X_model.columns), "excluded": ["senior_citizen (fairness
monitoring only)"],
        "test_metrics": {"roc_auc": round(roc_auc_score(yte, p_test), 3), "pr_auc":
round(average_precision_score(yte, p_test), 3)},
        "operating_point": "top 500 customers per month by probability", "intended_use":
"monthly retention targeting",
        "not_for": ["pricing", "credit decisions", "denying service"], "monitoring":
["monthly PSI of inputs", "AUC on matured labels", "holdout churn vs treated churn"],
        "retraining": "quarterly or when PSI > 0.25 / AUC drops by 0.03", "owner": "Data
Science, Retention analytics"}
print(json.dumps(card, indent=1)[:900])
```

OUTPUT

```
{
 "name": "Telecom churn propensity",
 "version": "1.0",
 "model": "logistic regression",
```

```
 "sklearn": "1.9.1",
 "features": [
  "tenure_months",
  "contract",
  "internet_service",
  "payment_method",
  "monthly_charges",
  "total_charges",
  "support_calls",
  "is_new",
  "charge_per_tenure_month",
  "price_change_proxy",
  "high_support",
  "m2m_and_new"
 ],
 "excluded": [
  "senior_citizen (fairness monitoring only)"
 ],
 "test_metrics": {
  "roc_auc": 0.793,
  "pr_auc": 0.579
 },
 "operating_point": "top 500 customers per month by probability",
 "intended_use": "monthly retention targeting",
 "not_for": [
  "pricing",
  "credit decisions",
  "denying service"
 ],
 "monitoring": [
  "monthly PSI of inputs",
  "AUC on matured labels",
  "holdout churn vs treated churn"
 ],
 "retraining": "quarterly or when PSI > 0.25 / AUC drops by 0.03",
 "owner": "Data Science, Retention analytics"
```


Deployment: a monthly batch job (Python + scheduler or Airflow) reads the latest customer table from the warehouse, scores it with the saved pipeline, writes `customer_id, p_churn, reason, action, model_version` to a table, which feeds the CRM and a Power BI dashboard tracking campaign results.

9. Executive summary

KEY TAKEAWAYS

- **Problem:** about 26% of customers churn. The retention budget covers 500 contacts a month.
- **Model:** a churn-risk model with ROC-AUC around 0.8 on unseen customers, explainable by tenure, contract type, support calls and charges.
- **Targeting:** contacting the top-ranked 500 reaches roughly twice as many churners as random selection, with a positive expected return per contact.
- **Actions:** service recovery for frequent complainers, an annual-plan upgrade for new month-to-month customers, and fibre quality checks.
- **Next:** run the 500 vs 100 holdout experiment for 60 days, then scale the budget if the incremental ROI holds.

Portfolio checklist

- README with problem, data, approach, results (numbers), business recommendation and how to run.
- Notebook → clean `src/` modules (features, train, evaluate, score) + tests for feature functions.
- Saved pipeline, model card, and a chart of profit vs number of contacts.
- A two-minute video or slide deck walking through the decision.

Summary

KEY TAKEAWAYS

- Start from the decision and economics, not the algorithm; define target, unit, metric, baseline and constraints.
- Build leak-free features in a pipeline; compare baselines and models with cross-validation; tune; test once.
- Prefer simpler models when performance is similar; check calibration before using probabilities in profit maths.
- Choose the operating point from expected profit and capacity; attach reasons and tailored actions.
- Measure real impact with a randomised holdout, document with a model card, and plan monitoring and retraining.

Quick quiz

6 questions · tap an answer to check it

1. What decision does the churn model support?

A Which customers receive the limited retention offer each month

B How to price new plans

C Whether to hire more agents

D Which features to collect

Answer: A. Always tie the model to a decision.

2. Why use a randomised holdout after deployment?

A The model predicts churn risk, not the causal effect of the offer; only an experiment measures incremental saves

B To retrain the model

C To check calibration

D To satisfy SQL

Answer: A. Measure uplift, not just accuracy.

3. Contact cost ₹300, value ₹4,000, acceptance 35%. Above which churn probability does a contact pay off (if calibrated)?

A 0.5

B About 0.21

C 0.05

D 0.9

Answer: B. 300 / (0.35 × 4,000).

4. Two models have CV AUC 0.812 and 0.815. Which do you choose?

A Usually the simpler, more explainable one

B Always the higher one

C Neither

D The one with more features

Answer: A. Small differences are within noise.

5. Why exclude `senior_citizen` from targeting features but keep it for monitoring?

A To avoid targeting on a sensitive attribute while still checking outcomes for fairness

B It has missing values

C It is a duplicate

D It has no signal

Answer: A. Fairness by design and by audit.

6. When should the test set be used?

A Once, after all modelling choices are final

B For tuning hyperparameters

C At every step

D Never

Answer: A. Otherwise the estimate is optimistic.

EXERCISE

Extend the capstone

(1) Replace the probability ranking with an expected-profit ranking that also uses each customer's monthly charges as their value. (2) Add SHAP reasons for each targeted customer (lesson 27). (3) Simulate the holdout experiment assuming the offer cuts churn by 35% among treated churners and test significance. (4) Write the README for GitHub.

Show solution (worked answer)

Python

```
val = tc.loc[Xte.index, "monthly_charges"].to_numpy() * 24              # ≈ 2 years of
revenue
exp_profit = p_test * accept * val - cost
top = np.argsort(-exp_profit)[:125]; print(int(exp_profit[top].sum()), yte.to_numpy()
[top].sum())
rng = np.random.default_rng(1); churn_t = rng.random(500) < 0.55 * 0.65; churn_c =
rng.random(100) < 0.55
from statsmodels.stats.proportion import proportions_ztest
print(churn_t.mean(), churn_c.mean(), proportions_ztest([churn_t.sum(),
churn_c.sum()], [500, 100]))
```

INTERVIEW CORNER

Questions you may be asked

Q: Walk me through an end-to-end ML project. Problem framing and economics, data and leak-free features, EDA, baselines, cross-validated model comparison and tuning, one-time test evaluation, threshold selection from business value, explanation and fairness checks, deployment as a batch or API, experiment to measure impact, and monitoring with retraining.

Q: Your model has AUC 0.80. Is that good? It depends on the decision: compare with the baseline rule, look at precision/recall in the top N the business can act on, and translate it into expected profit; AUC alone does not say.

Q: How do you know the retention campaign worked? Randomly hold out some high-risk customers from the offer and compare churn between treated and holdout groups; the difference (with a confidence interval) is the causal effect.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.